Classification of modulo-q total-search classes by prime factors via coprime and prime-power reductions

Mika Göös, Pritish Kamath,
Katerina Sotiraki, Manolis Zampetakis

doi:10.4230/LIPIcs.CCC.2020.19; CC BY 3.0 Unported

Editorial scope. One independent composite-modulus classification mechanism,grouping coprime and prime-power reductions with their support. Operator& is the tagged-input class combination. Concurrent independent Hollender result attributed and not counted separately. No CWT/SIS/application/survey tasks split from this theorem.

Allowed standard prerequisites (editorial).

- TFNP many-one reductions,prime divisibility of binomial coefficients and efficient subset indexing. Exact formal q>=2 and prime-p conventions retained;operator& defined through tagged-input complete problems,not confused with set intersection. Fact11 binomial congruences and cited efficient subset indexing[31] remain standard prerequisites;full local canonical grouping/indexing/witness arguments and graph equivalences retained. Full37 original references.

1.1 Characterization via Prime Modulus

We show, in section 3, that every class PPA_q is built out of the classes PPA_p for p a prime. To formalize this result, we recall the operator '&' defined by Buss and Johnson [13, §6]. For any two syntactic complexity classes M_0, M_1 with complete problems S_0, S_1, the class M_0 & M_1 is defined via its complete problem S_0 & S_1 where, on input $(x, b) \in \{0,1\}^* \times \{0,1\}$, the goal is to find a solution for x interpreted as an instance of problem S_b. Namely, if $b = 0$ then the output has to be a solution of S_0 with input x, and otherwise it has to be a solution of S_1 with input x. Intuitively speaking, M_1 & M_2 combines the powers of both M_1 and M_2. Note that $\mathsf{M}_1 \cup \mathsf{M}_2 \subseteq \mathsf{M}_1$ & M_2. We can now formally express our characterization result (where $p|q$ is the set of primes p dividing q).

Theorem 1. $\mathsf{PPA}_q = \&_{p|q}\, \mathsf{PPA}_p$.

A special case of Theorem 1 is that $\mathsf{PPA}_{p^k} = \mathsf{PPA}_p$ for every prime power p^k. Showing the inclusion $\mathsf{PPA}_{p^k} \subseteq \mathsf{PPA}_p$ is the crux of our proof. This part of the theorem can be viewed as a total search problem analog of the counting class result of Beigel and Gill [7] stating that $\mathsf{Mod}_{p^k}\mathsf{P} = \mathsf{Mod}_p\mathsf{P}$; "an unexpected

result", they wrote at the time. Throughout this paper, we use q to denote any integer ≥ 2 and p to denote a prime integer.

Concurrent independent work (original attribution) We note that a concurrent and independent work by Hollender [25] also establishes the structural properties corresponding to the prime-modulus characterization and relative structural-power discussion.

2 The class PPA_q

Search Problems in FNP and TFNP. A search problem in FNP is defined by a polynomial time computable relation $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^*$, that is, for every (x, y), it is possible to decide whether $(x, y) \in \mathcal{R}$ in $\text{poly}(|x|, |y|)$ time. A solution to the search problem on input x is a y such that $|y| = \text{poly}(|x|)$ and $(x, y) \in \mathcal{R}$. For convenience, define $\mathcal{R}(x) \coloneqq \{y : (x, y) \in \mathcal{R}\}$. A search problem is *total* if for every input $x \in \{0,1\}^*$, there exists $y \in \mathcal{R}(x)$ such that $|y| \leq \text{poly}(|x|)$. TFNP is the class of all total search problems in FNP.

Reducibility among search problems. A search problem $\mathcal{R}_1$ is *Karp-reducible* (or *many-one reducible*) to a search problem $\mathcal{R}_2$, or $\mathcal{R}_1 \preceq \mathcal{R}_2$ for short, if there exist polynomial-time computable functions f and g such that given any instance x of $\mathcal{R}_1$, $f(x)$ is an instance of $\mathcal{R}_2$ such that for any $y \in \mathcal{R}_2(f(x))$, it holds that $g(x, f(x), y) \in \mathcal{R}_1(x)$.

On the other hand, we say that $\mathcal{R}_1$ is *Turing-reducible* to $\mathcal{R}_2$, or $\mathcal{R}_1 \preceq_T \mathcal{R}_2$ for short, if there exists a polynomial-time oracle Turing machine that on input x to $\mathcal{R}_1$, makes oracle queries to $\mathcal{R}_2$, and outputs a $y \in \mathcal{R}_1(x)$. In this paper, we primarly deal with Karp-reductions, except in Section7.3, where we compare the two different notions of reductions in the context of PPA_q.

PPA_q via complete problems. We describe several total search problems (parameterized by q) that we show to be inter-reducible. PPA_q is then defined as the set of all search problems reducible to either one of the search problems defined below.

Recall that Boolean circuits take inputs of the form $\{0,1\}^n$ and operate using $(\wedge, \vee, \neg)$ gates. In addition, we'll also consider circuits acting on inputs in $[q]^n$. We interpret the input to be of the form $(\{0,1\}^{\lceil \log q \rceil})^n$, where the circuit will be evaluated only on inputs where each block of $\lceil \log q \rceil$ bits represents a element in $[q]$. In the case where q is a prime, we could also represent the circuit as $C : \mathbb{F}_q^n \to \mathbb{F}_q^n$ with arbitrary gates of the form $g : \mathbb{F}_q^2 \to \mathbb{F}_q$. However, we can simulate any such gate with $\text{poly}(q)$ many $+$ and $\times$ operations (over $\mathbb{F}_q$) along with a constant (1) gate. Hence, in the case of prime q, we'll assume that such circuits are composed of only $(+, \times, 1)$ gates.

Definition 6. ($\textsc{Bipartite}_q$)

Principle: A bipartite graph with a non-multiple-of-q degree node has another such node.

Object: Bipartite graph $G = (V \cup U, E)$. Designated vertex $v^* \in V$

Inputs: ▷ $C : \{0,1\}^n \to (\{0,1\}^n)^k$, with $(\{0,1\}^n)^k$ interpreted as a k-subset of $\{0,1\}^n$
▷ $v^* \in \{0\} \times \{0,1\}^{n-1}$ (usually 0^n)

Encoding: $V \coloneqq \{0\} \times \{0,1\}^{n-1}$, $U \coloneqq \{1\} \times \{0,1\}^{n-1}$,
$E \coloneqq \{(v,u) : v \in V \cap C(u) \text{ and } u \in U \cap C(v)\}$

Solutions: v^* if $\deg(v^*) \equiv 0 \pmod q$ and
$v \neq v^*$ if $\deg(v) \not\equiv 0 \pmod q$

Definition 7. ($\textsc{Lonely}_q$)

Principle: A q-dimensional matching on a non-multiple-of-q many vertices has an isolated node.

Object: q-dimensional matching $G = (V, E)$. Designated vertices $V^* \subseteq V$ with $|V^*| \leq q - 1$

Inputs: ▷ $C : [q]^n \to [q]^n$
▷ $V^* \subseteq [q]^n$ with $|V^*| \leq q - 1$

Encoding: $V \coloneqq [q]^n$. For distinct $v_1, \ldots, v_q$, edge $e \coloneqq \{v_1, \ldots, v_q\} \in E$ if $C(v_i) = v_{i+1}$, $C(v_q) = v_1$

Solutions: $v \in V^*$ if $\deg(v) = 1$ and
$v \notin V^*$ if $\deg(v) = 0$

Definition 8. ($\textsc{Leaf}_q$)

Principle: A q-uniform hypergraph with a non-multiple-of-q degree node has another such node.

Object: q-uniform hypergraph $G = (V, E)$. Designated vertex $v^* \in V$

Inputs: ▷ $C : \{0,1\}^n \to (\{0,1\}^{nq})^q$, with $(\{0,1\}^{nq})^q$ interpreted as q many q-subsets of $\{0,1\}^n$
▷ $v^* \in \{0,1\}^n$ (usually 0^n)

Encoding: $V \coloneqq \{0,1\}^n$. For distinct $v_1, \ldots, v_q$, edge $e \coloneqq \{v_1, \ldots, v_q\} \in E$ if $e \in C(v)$ for all $v \in e$

Solutions: v^* if $\deg(v) \equiv 0 \pmod q$ and
$v \neq v^*$ if $\deg(v) \not\equiv 0 \pmod q$

We remark that $\textsc{Lonely}_q$ and $\textsc{Leaf}_q$ are modulo-q analogs of the PPA-complete problems Lonely and Leaf [33, 5]. We prove the following theorem in Appendix A.

Theorem 9. *The problems $\textsc{Bipartite}_q$, $\textsc{Lonely}_q$ and $\textsc{Leaf}_q$ are inter-reducible.*

Remark 10 (Simplifications in describing reductions.). We will use the following simple conventions repeatedly, in order to simplify the descriptions of reductions between different search problems.

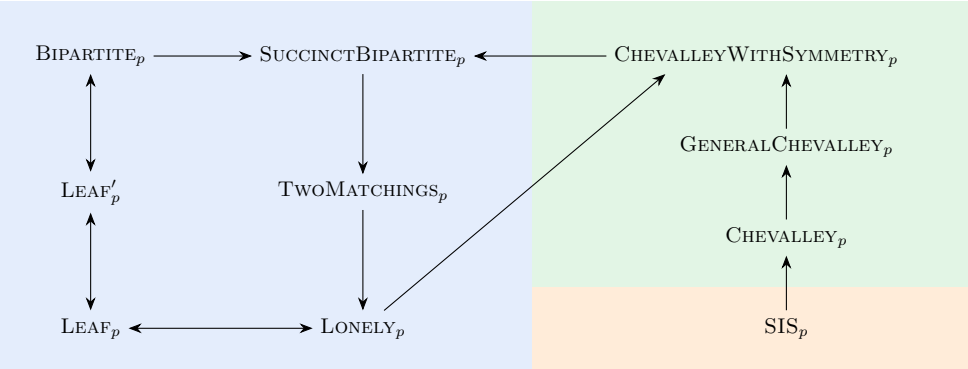


Figure 2: Total search problems studied in this work. An arrow $A \to B$ denotes a reduction $A \preceq B$ that we establish. Problems in the blue region are non-natural problems, which are all complete for PPA_p. Problems in the green region are natural problems of which ChevalleyWithSymmetry$_p$ is the one we show to be PPA_p–complete. The problem in the orange region is a cryptographically relevant problem.

1. We will often use "algorithms", instead of "circuits" to encode our hypergraphs. It is standard to simulate polynomial-time algorithms by polynomial sized circuits.
2. While our definitions require vertex sets to be of a very special form, e.g. $\{0,1\}^n$ or $[q]^n$, it will hugely simplify the description of our reductions to let vertex sets be of arbitrary sizes. This is not a problem as long as the vertex set is efficiently indexable, that is, elements of V must have a poly(n) length representation and we must have a poly-time computable bijective map $\varphi : V \to [|V|]$, whose inverse is also poly-time computable. We could then use φ to interpret the first $|V|$ elements of $\{0,1\}^n$ (or $[q]^n$) as vertices in V.
   Note that, we need to ensure that no new solutions are introduced in this process. In the case of Bipartite$_q$ or Leaf$_q$, we simply leave the additional vertices isolated and they don't contribute any new solutions. In the case of Lonely$_q$ we need to additionally ensure that $|V| \equiv 0 \pmod q$, so that we can easily partition the remaining vertices into q-uniform hyperedges thereby not introducing any new solutions.
3. The above simplification gives us that all our problems have an *instance-extension property* (cf. [10]) – this will be helpful in proving Theorem5.
4. To simplify our reductions even further, we'll often describe the edges/hyperdges directly instead of specifying how to compute the neighbors of a given vertex. This is only for simplicity and it will be easy to see how to compute the neighbors of any vertex locally.

3 Characterization via Primes

In this section we prove Theorem 1, namely $\mathsf{PPA}_q = \&_{p|q}\, \mathsf{PPA}_p$. The theorem follows by combining the following two ingredients.

§3.1: $\mathsf{PPA}_{qr} = \mathsf{PPA}_q$ & PPA_r for any coprime q and r.
§3.2: $\mathsf{PPA}_{p^k} = \mathsf{PPA}_p$ for any prime power p^k.

3.1 Coprime case

$\mathsf{PPA}_{qr} \supseteq \mathsf{PPA}_q$ & PPA_r. We show that $\textsc{Lonely}_q$ & $\textsc{Lonely}_r$ reduces to $\textsc{Lonely}_{qr}$. Recall that an instance of $\textsc{Lonely}_q$ & $\textsc{Lonely}_r$ is a tuple (C, V^*, b) where (C, V^*) describes an instance of either $\textsc{Lonely}_q$ or $\textsc{Lonely}_r$ as chosen by $b \in \{0,1\}$. Suppose wlog that $b = 0$, so the input encodes a q-dimensional matching $G = (V, E)$ over $V = [q]^n$ with designated vertices $V^* \subseteq V$, $|V^*| \not\equiv 0 \pmod q$. We can construct a qr-dimensional matching $\overline{G} = (\overline{V}, \overline{E})$ on vertices $\overline{V} \coloneqq V \times [r]$ as follows: For every hyperedge $e \coloneqq \{v_1, \ldots, v_q\} \in E$, we include the hyperedge $e \times [r]$ in $\overline{E}$. We let the designated vertices of $\overline{G}$ be $\overline{V}^* \coloneqq V^* \times [r]$. Note that $|\overline{V}^*| \not\equiv 0 \pmod{qr}$. It is easy to see that a vertex (v, i) is isolated in G' iff v is isolated in G. This completes the reduction since $\overline{V}$ is efficiently indexable, and the neighbors of any vertex in $\overline{V}$ are locally computable using black-box access to C.

$\mathsf{PPA}_{qr} \subseteq \mathsf{PPA}_q$ & PPA_r. We show that $\textsc{Bipartite}_{qr}$ reduces to $\textsc{Bipartite}_q$ & $\textsc{Bipartite}_r$. Our input instance of $\textsc{Bipartite}_{qr}$ is a circuit $C : \{0,1\}^n \to (\{0,1\}^n)^k$ that encodes a bipartite graph $G = (V \cup U, E)$ with a designated node $v^* \in V$. If $\deg(v^*) \equiv 0 \pmod{qr}$, then we already have solved the problem and no further reduction is necessary. Otherwise, if $\deg(v^*) \not\equiv 0 \pmod{qr}$, we have, by the coprime-ness of q and r, that either $\deg(v^*) \not\equiv 0 \pmod q$ or $\deg(v^*) \not\equiv 0 \pmod r$. In the first case (the second case is analogous), we can simply view (G, v^*) as an instance of $\textsc{Bipartite}_q$, since vertices with degree $\not\equiv 0 \pmod q$ in G are also solutions to $\textsc{Bipartite}_{qr}$.

3.2 Prime power case

$\mathsf{PPA}_{p^k} \supseteq \mathsf{PPA}_p$ follows immediately from our proof of $\mathsf{PPA}_{qr} \supseteq \mathsf{PPA}_q$ & PPA_r, which didn't require that q and r be coprime. It remains to show $\mathsf{PPA}_{p^k} \subseteq \mathsf{PPA}_p$. We exploit the following easy fact.

Fact 11. *For all primes p, it holds that,*

$$\text{for integers } t, c > 0: \quad \binom{c \cdot p^t}{p^t} \equiv 0 \pmod p \quad \text{if and only if} \quad c \equiv 0 \pmod p \tag{1}$$

$$\text{for integer } k > 0: \quad \binom{p^k}{i} \equiv 0 \pmod p \quad \text{for all} \quad 0 < i < p^k \tag{2}$$

We reduce $\textsc{Lonely}_{p^k}$ to $\textsc{Lonely}_p$. Our instance of $\textsc{Lonely}_{p^k}$ is (C, V^*) where C implicitly encodes a p^k-dimensional matching $G = (V = [p^k]^n, E)$ and a designated vertex set $V^* \subseteq V$ such that $|V^*| \not\equiv 0 \pmod{p^k}$.

Let p^t, $0 \le t < k$, be the largest power of p that divides $|V^*|$. Through local operations we construct a p-dimensional matching hypergraph $\overline{G} = (\overline{V}, \overline{E})$ over vertices $\overline{V} \coloneqq \binom{V}{p^t}$ (set of all size-p^t subsets of V) with designated vertices $\overline{V}^* \coloneqq \binom{V^*}{p^t}$. From Eq. 1, we get that $|\overline{V}| \equiv 0 \pmod p$ and $|\overline{V}^*| \not\equiv 0 \pmod p$.

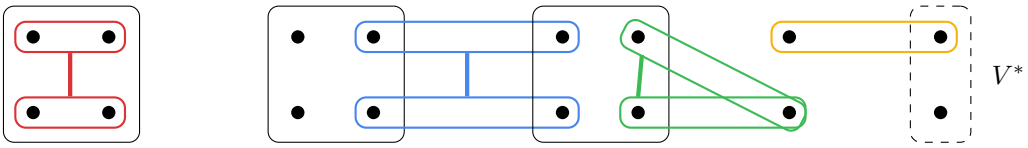


Figure 3: Illustration of the proof of $\mathsf{PPA}_{p^k} \subseteq \mathsf{PPA}_p$ for $p = 2$, $k = 2$, $n = 2$, $t = 1$. In black, we indicate the 4-dimensional matching G. In color, we highlight some of the vertices of $\overline{G}$ and the edges between them. The vertices of $\overline{G}$ in red, blue and green are paired up and hence are non-solutions; whereas the vertex in yellow is isolated and not in $\overline{V}^*$ and hence a solution.

We will describe an algorithm that on vertex $\overline{v} \in \overline{V}$ outputs a hyperedge of p vertices that contains $\overline{v}$ (if any). To this end, first fix an algorithm that for any set $e \coloneqq \{u_1, \ldots, u_{p^k}\} \subseteq V$ and for any $1 \leq i \leq p^t$, computes some "canonical" partition of the set $\binom{e}{i}$ into subsets of size p, and moreover assigns a canonical cyclic order within each such subset. This is indeed possible because of Eq. 2, since $t < k$.

Given a vertex $\overline{v} \coloneqq \{v_1, \ldots, v_{p^t}\} \in \overline{V}$,

- ▷ Compute all edges $e_1, \ldots, e_\ell \in E$ that include some $v \in \overline{v}$.
- ▷ For edge e_j, define $S_j \coloneqq e_j \cap \overline{v}$ and let $S_j^1, \ldots, S_j^{p-1}$ be the remaining subsets in the same partition as S_j in the canonical partition of $\binom{e_j}{|S_j|}$, listed in the canonical cyclic order starting at S_j. Also, let S_0 be the set of untouched vertices in $\overline{v}$. Observe that $\overline{v} = S_0 \cup S_1 \cup \ldots \cup S_\ell$.
- ▷ Output neighbors of $\overline{v}$ as the vertices $\overline{v}_1, \ldots, \overline{v}_{p-1}$ where $\overline{v}_i \coloneqq S_0 \cup S_1^i \cup \ldots \cup S_\ell^i$.

It is easy to see that $\overline{v}$ is isolated in $\overline{G}$ iff all $v \in \overline{v}$ are isolated in G. Moreover, any isolated vertex in $\overline{V} \setminus \overline{V}^*$ contains at least one isolated vertex in $V \setminus V^*$; and a non-isolated vertex in $\overline{V}^*$ contains at least one non-isolated vertex in V^* (in fact p^t many).

The edges of $\overline{G}$ can indeed be computed efficiently with just black-box access to C. In order to complete the reduction, we only need that $\overline{V}$ is efficiently indexable. This is indeed standard; see [31, §2.3] for a reference. See Figure 3 for an illustration of the proof.

Remark 12. Note that the size of the underlying graph blows up polynomially in our reduction. We do not know whether a reduction exists that avoids such a blow-up, although we suspect that the techniques of [6] can be used to show that some blow-up is necessary for black-box reductions.

A Appendix: Reductions Between Complete Problems

In order to prove Theorem 9, we introduce an additional problem that will serve as intermediate problem in our reductions.

Definition 47. ($\textsc{Leaf}'_q$)

Principle: Same as $\textsc{Leaf}_q$, but degrees are allowed to be larger (polynomially bounded).

Object: q-uniform hypergraph $G = (V, E)$. Designated vertex $v^* \in V$.

Inputs: ▷ $C : \{0,1\}^n \to (\{0,1\}^{nq})^k$, with $(\{0,1\}^{nq})^k$ interpreted as k many q-subsets of $\{0,1\}^n$
▷ $v^* \in \{0,1\}^n$ (usually 0^n)

Encoding: $V \coloneqq \{0,1\}^n$.
For distinct $v_1, \ldots, v_q$, edge $e \coloneqq \{v_1, \ldots, v_q\} \in E$ if $e \in C(v)$ for all $v \in e$

Solutions: v^* if $\deg(v) \equiv 0 \pmod q$ and
$v \neq v^*$ if $\deg(v) \not\equiv 0 \pmod q$

Proof of Theorem 9. We show the following inter-reducibilities: (1) $\textsc{Leaf}_q \asymp \textsc{Leaf}'_q$, (2) $\textsc{Leaf}'_q \asymp \textsc{Bipartite}_q$ and (3) $\textsc{Leaf}_q \asymp \textsc{Lonely}_q$.

(1a) $\textsc{Leaf}_{\boldsymbol{q}} \preceq \textsc{Leaf}'_{\boldsymbol{q}}$ Each instance of $\textsc{Leaf}_q$ is trivially an instance of $\textsc{Leaf}'_q$.

(1b) $\textsc{Leaf}'_{\boldsymbol{q}} \preceq \textsc{Leaf}_{\boldsymbol{q}}$. We start with a $\textsc{Leaf}'_q$ instance (C, v^*), where C encode a q-uniform hypergraph $G = (V, E)$ with degree at most k. Let $t = \lceil k/q \rceil$. We construct a $\textsc{Leaf}_q$ instance encoding a hypergraph $\overline{G} = (\overline{V}, \overline{E})$ on vertex set $\overline{V} \coloneqq V \times [t]$, intuitively making t copies of each vertex.

In order to locally compute hyperedges, we first fix a canonical algorithm that for any vertex v and any edge $e \in E$ incident on v, assigns it a label $\ell_v(e) \in [t]$, with at most q edges mapping to the same label — e.g. sort all edges incident on v in lexicographic order and bucket them sequentially in at most t groups of at most q each. Note that we can ensure that for any vertex v at most one label gets mapped to by a non-zero, non-q number of edges. Moreover, if $\deg(v) \equiv 0 \pmod q$, then exactly q or 0 edges are assigned to any label.

We'll assume that $\deg(v^*) \not\equiv 0 \pmod q$, as otherwise, a reduction wouldn't be necessary. We let (v^*, ℓ^*) be the designated vertex of the $\textsc{Leaf}_q$ instance, where ℓ^* is the unique label that gets mapped to by a non-zero, non-q number of edges incident on v^*.

For any vertex $(v, i) \in \overline{V}$, we assign it at most q edges as follows: For each edge $e = \{v_1, \ldots, v_q\}$ such that $\ell_v(e) = i$, the corresponding hyperedge of (v, i) is $(v_1, \ell_{v_1}(e)), \ldots, (v_q, \ell_{v_q}(e))$. It is easy to see that the designated vertex (v^*, ℓ^*) indeed has non-zero, non-q degree. Moreover, a vertex $\deg(v, i) \notin \{0, q\}$ in $\overline{G}$ only if v has a non-multiple-of-q degree in G. Thus, solutions to the $\textsc{Leaf}_q$ instance naturally maps to solutions to the original $\textsc{Leaf}'_q$ instance.

By Remark 10, this completes the reduction since the edges are locally computable with black-box access to C and $\overline{V}$ is efficiently indexable.

(2a) $\textsc{Leaf}'_{\boldsymbol{q}} \preceq \textsc{Bipartite}_{\boldsymbol{q}}$. We start with a $\textsc{Leaf}'_q$ instance (C, v^*), where C encode a q-uniform hypergraph $G = (V, E)$. We construct a $\textsc{Bipartite}_q$ instance encoding a graph $\overline{G} = (\overline{V} \cup \overline{U}, \overline{E})$ such that $\overline{V} = V$ and $\overline{U} = \binom{V}{q}$, i.e. all q-sized subsets of V. We include the edge $(v, e) \in \overline{E}$ if $e \in E$ is incident on v. The designated vertex for the $\textsc{Bipartite}_q$ instance is v^* in $\overline{V}$.

Clearly, all vertices $e \in \overline{U}$ have degree either q or 0. For any $v \in \overline{V}$, the degree of v in $\overline{G}$ is same as its degree in G. Thus, any solution to the $\textsc{Bipartite}_q$ instance immediately gives a solution to the original $\textsc{Leaf}'_q$ instance. By Remark 10, this completes the reduction since the edges are locally computable with black-box access to C and $\overline{V}$ and $\overline{U}$ are efficiently indexable (cf. [31, §2.3] for efficiently indexing $\overline{U}$).

(2b) $\textsc{Bipartite}_{\boldsymbol{q}} \preceq \textsc{Leaf}'_{\boldsymbol{q}}$. We start with a $\textsc{Bipartite}_q$ instance (C, v^*) encoding a bipartite graph $\mathcal{G} = (V \cup U, E)$ with maximum degree of any vertex being at most k. We construct a $\textsc{Leaf}'_q$ instance encoding a hypergraph $\overline{G} = (\overline{V}, \overline{E})$ such that $\overline{V} = V$ with designated vertex v^*.

First, we fix a canonical algorithm that for any vertex $u \in U$ with $\deg_G(u) \equiv 0 \pmod q$ produces a partition of it's neighbors with q vertices of V in each part. Now, the set of q-uniform hyperedges incident on any vertex $v \in \overline{V}$ in $\overline{E}$ can be obtained as: for all neighbors u of v, with $\deg_G(u) \equiv 0 \pmod q$, we include a hyperedge consisting of all vertices in the same partition as v among the neighbors of u (we ignore neighbors u with $\deg(u) \not\equiv 0 \pmod q$).

Observe that $\deg_{\overline{G}}(v) \leq \deg_G(v)$ and equality holds if and only if all neighbors of v in G have degree $\equiv 0 \pmod q$. Hence for any $v \in \overline{V}$, if $\deg_{\overline{G}}(v) \neq \deg_G(v) \pmod q$, then there exists a neighbor $u \in U$ of v in G such that $\deg(u) \not\equiv 0 \pmod q$. Thus, if $v = v^*$ and $\deg_{\overline{G}}(v^*) \equiv 0 \pmod q$, then either $\deg_G(v) \equiv 0 \pmod q$ or we can find a neighbor u of v in G with $\deg(u) \not\equiv 0 \pmod q$. Similarly if for some $v \neq v^*$, we have $\deg_{\overline{G}}(v^*) \not\equiv 0 \pmod q$, then either $\deg_G(v) \not\equiv 0 \pmod q$ or we can find a neighbor u of v in G with $\deg(u) \not\equiv 0 \pmod q$. Thus, any solution to the $\textsc{Leaf}'_q$ instance gives us a solution to the original $\textsc{Bipartite}_q$ instance. This completes the reduction since $\overline{V} = \{0,1\}^n$ and the edges are locally computable with black-box access to C.

(3a) $\textsc{Leaf}_{\boldsymbol{q}} \preceq \textsc{Lonely}_{\boldsymbol{q}}$. We start with a $\textsc{Leaf}_q$ instance (C, v^*), where C encode a q-uniform hypergraph $G = (V, E)$ with degree at most q. If $\deg_G(v^*) = q$ or 0, then we don't need any further reduction. Else, we construct a $\textsc{Lonely}_q$ instance encoding a q-dimensional matching $\overline{G} = (\overline{V}, \overline{E})$ on vertex set $\overline{V} = V \times [q]$. The designated vertices will be $V^* = \{(v, q-i) : 1 \leq i \leq q - \deg(v^*)\}$. Note that, $|V^*| = q - \deg_G(v^*)$ and hence $1 \leq |V^*| \leq q-1$.

In order to locally compute hyperedges, we first fix a canonical algorithm that for any vertex v and any edge $e \in E$ incident on v, assigns it a unique label $\ell_v(e) \in [q]$ — e.g. sort all edges incident on v in lexicographic order and label them sequentially in $[q]$. In fact, we can ensure that an edge incident on v get labeled within $\{1, \ldots, \deg_G(v)\}$.

For any vertex $(v, i) \in \overline{V}$, we assign it at most one hyperedge as follows:

- ▷ If $\deg_G(v) = 0$, we include the hyperedge $\{(v, i) : i \in [q]\}$.
- ▷ Else if $\deg_G(v) \geq i$, then for edge $e = \{v_1, \ldots, v_q\}$ incident on v such that $\ell_v(e) = i$, the corresponding hyperedge of (v, i) is $(v_1, \ell_{v_1}(e)), \ldots, (v_q, \ell_{v_q}(e))$.
- ▷ Else if $0 < \deg_G(v) < i$, we leave it isolated.

It is easy to see that our definition of hyperedges is consistent and that the designated vertices V^* are indeed isolated. Moreover, a vertex (v, i) is isolated in $\overline{G}$ only if $1 \leq \deg_G(v) \leq q-1$. Thus, solutions to the $\textsc{Leaf}_q$ instance naturally maps to solutions to the original $\textsc{Leaf}'_q$ instance.

By Remark 10, this completes the reduction since the edges are locally computable with black-box access to C and $\overline{V}$ is efficiently indexable.

(3b) $\textsc{Lonely}_{\boldsymbol{q}} \preceq \textsc{Leaf}_{\boldsymbol{q}}$. We start with a $\textsc{Lonely}_q$ instance (C, V^*), where C encode a q-dimensional matching $G = (V, E)$. We construct a $\textsc{Leaf}_q$ instance encoding a q-uniform hypergraph $\overline{G} = (\overline{V}, \overline{E})$ on vertex set $\overline{V}$ that will be specified shortly. We describe the hyperedges in $\overline{G}$ and it'll be clear how to compute the hyperedges for any vertex locally with just black-box access to C.

We start with $\overline{V} = V$. Our goal is to transform all vertices of degree 1 to degree q, while ensuring that vertices of degree 0 are mapped to vertices of degree not a multiple of q. Towards this goal we let $\overline{E}$ to be set of edges in E in addition to $q-1$ canonical q-dimensional matchings over V. For example, for a vertex $v \coloneqq (x_1, \ldots, x_n) \in V = [q]^n$, the corresponding edges in $\overline{E}$ include an edge in E (if any) and edges of the type $e_i = \{(x_1, \ldots, x_{i-1}, j, x_{i+1}, \ldots, x_n) : j \in [q]\}$ for $i \in [q-1]$ (note, this requires us to assume $n \geq q-1$). Adding the $q-1$ matchings increases the degree of each vertex by $q-1$. Therefore, vertices with initial degree 1 now have degree q and vertices with initial degree 0 now have degree $q-1$. However, a couple of issues remain in order to complete the reduction, which we handle next.

Multiplicities. An edge $e \in E$ might have gotten added twice, if it belonged to one of the canonical matchings. To avoid this issue altogether, instead of adding edges directly on V, we augment $\overline{V}$ to become $\overline{V} \coloneqq V \cup \left(\binom{V}{q} \times [q-1]\right)$, i.e. in addition to V, we have $q-1$ vertices for every potential hyperedge of G. For any edge $e \coloneqq \{v_1, \ldots, v_q\} \in E$, instead of adding it directly in $\overline{G}$, we add hyperedge $\{v, (e,1), (e,2), \ldots, (e, q-1)\}$ for each $v \in e$. Note that, all vertices $(e, i) \in \binom{V}{q} \times [q-1]$ have degree q if $e \in E$ and degree 0 if $e \notin E$, so they are non-solutions for the $\textsc{Leaf}_q$ instance. For vertices in V, we still have as before that vertices with initial degree 1 now have degree q and vertices with initial degree 0 now have degree $q-1$.

Designated vertex. In a $\textsc{Leaf}_q$ instance, we need to specify a single designated vertex $v^* \in \overline{V}$. If the $\textsc{Lonely}_q$ instance had a single designated vertex then we would be done. However, in general it is not possible to assume this (for non-prime q). Nevertheless, we provide a way to get around this. We augment $\overline{V}$ with $t = (q-1)(q-k)+1$ additional vertices to become $\overline{V} \coloneqq V \cup \left(\binom{V}{q} \times [q-1]\right) \cup \{w_{i,j} : i \in [q-k],\ j \in [q-1]\} \cup \{v^*\}$, where v^* will eventually be the single designated vertex for the $\textsc{Leaf}_q$ instance.

Let $V^* = \{u_1, \ldots, u_k\} \subseteq V$ be the set of designated vertices in the $\textsc{Lonely}_q$ instance (note $1 \leq k < q$). So far, note that $\deg_{\overline{G}}(u_i) = q-1$. The only new hyperedges we add will be among u_i's, $w_{i,j}$'s and v^*, in such a way that $\deg_{\overline{G}}(u_i)$ will become q, the degree of all $w_{i,j}$'s will also be q and degree of v^* will be $q-k$.

- ▷ For each $u \in V^*$, include $\{u, w_{1,1}, \ldots, w_{1,q-1}\}$. So far, $\deg_{\overline{G}}(u) = q$ and $\deg_{\overline{G}}(w_{1,j}) = k$.
- ▷ For each $j \in [q-1]$ and each $i \in \{2, \ldots, q-k\}$, include $\{w_{1,j}, w_{i,1}, \ldots, w_{i,q-1}\}$. So far, $\deg_{\overline{G}}(w_{i,j}) = q-1$ for all $(i,j) \in [q-k] \times [q-1]$.
- ▷ Finally, for each $(i,j) \in [q-k] \times [q-1]$, include $\{v^*, w_{i,1}, \ldots, w_{i,q-1}\}$. Now, $\deg_{\overline{G}}(w_{i,j}) = q$ for all $(i,j) \in [q-k] \times [q-1]$ and $\deg_{\overline{G}}(v^*) = q-k$.

Thus, we have finally reduced to a $\textsc{Leaf}_q$ instance encoding the graph $\overline{G} = (\overline{V}, \overline{E})$ with $\overline{V} \coloneqq V \cup \left(\binom{V}{q} \times [q-1]\right) \cup \{w_{i,j} : i \in [q-k],\ j \in [q-1]\} \cup \{v^*\}$. By Remark 10, this completes the reduction, since $\overline{V}$ is efficiently indexable (again,

see [31] for a reference on indexing $\binom{V}{q}$) and the edges are locally computable using black-box access to C. □

References

[1] James Aisenberg, Maria Luisa Bonet, and Sam Buss. 2-d tucker is PPA complete. Electronic Colloquium on Computational Complexity (ECCC), 22:163, 2015. URL: http://eccc.hpi-web. de/report/2015/163.

[2] Noga Alon. Splitting necklaces. Advances in Mathematics, 63(3):247–253, 1987. doi:10.1016/ 0001-8708(87)90055-7.

[3] Noga Alon, Shmuel Friedland, and Gil Kalai. Regular subgraphs of almost regular graphs. Journal of Combinatorial Theory, Series B, 37(1):79–91, 1984.

[4] Noga Alon and Douglas B. West. The Borsuk-Ulam theorem and bisection of necklaces. Proceedings of the American Mathematical Society, 98(4):623–628, 1986. doi: 10.1090/S0002-9939-1986-0861764-9.

[5] Paul Beame, Stephen A. Cook, Jeff Edmonds, Russell Impagliazzo, and Toniann Pitassi. The relative complexity of NP search problems. J. Comput. Syst. Sci., 57(1):3–19, 1998. doi:10.1006/jcss.1998.1575.

[6] Paul Beame and Søren Riis. More on the relative strength of counting principles. In Proceedings of the DIMACS Workshop on Proof Complexity and Feasible Arithmetics, volume 39, pages 13–35, 1998.

[7] Richard Beigel and John Gill. Counting classes: Thresholds, parity, mods, and fewness. Theor. Comput. Sci., 103(1):3–23, 1992. doi:10.1016/0304-3975(92)90084-S.

[8] Aleksandrs Belovs, Gábor Ivanyos, Youming Qiao, Miklos Santha, and Siyi Yang. On the polynomial parity argument complexity of the combinatorial nullstellensatz. In 32nd Computational Complexity Conference, CCC 2017, July 6-9, 2017, Riga, Latvia, pages 30:1– 30:24, 2017. doi:10.4230/LIPIcs.CCC.2017.30.

[9] Nir Bitansky, Omer Paneth, and Alon Rosen. On the cryptographic hardness of finding a nash equilibrium. In 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, pages 1480–1498. IEEE, 2015.

[10] Josh Buresh-Oppenheim and Tsuyoshi Morioka. Relativized NP search problems and propositional proof systems. In 19th Annual IEEE Conference on Computational Complexity (CCC 2004), 21-24 June 2004, Amherst, MA, USA, pages 54–67, 2004. doi: 10.1109/CCC.2004.1313795.

[11] Joshua Buresh-Oppenheim. On the TFNP complexity of factoring. Manuscript, 2006. URL: http://www.cs.toronto.edu/ bureshop/factor.pdf.

[12] Samuel R. Buss, Dima Grigoriev, Russell Impagliazzo, and Toniann Pitassi. Linear gaps between degrees for the polynomial calculus modulo distinct primes. J. Comput. Syst. Sci., 62(2):267–289, 2001. doi:10.1006/jcss.2000.1726.

[13] Samuel R. Buss and Alan S. Johnson. Propositional proofs and reductions between NP search problems. Ann. Pure Appl. Logic, 163(9):1163–1182, 2012. doi:10.1016/j.apal.2012.01.015.

[14] I. Bárány, S. B. Shlosman, and A. Szücs. On a topological generalization of a theorem of tverberg. Journal of the London Mathematical Society, s2-23(1):158–164, 1981. doi: 10.1112/jlms/s2-23.1.158.

[15] Claude Chevalley. Démonstration d'une hypothèse de m. artin. Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg, 11(1):73–75, December 1935. doi:10. 1007/BF02940714.

[16] Arka Rai Choudhuri, Pavel Hubácek, Chethan Kamath, Krzysztof Pietrzak, Alon Rosen, and Guy N Rothblum. Finding a nash equilibrium is no easier than breaking fiat-shamir. In Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, pages 1103–1114. ACM, 2019.

[17] Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a nash equilibrium. SIAM J. Comput., 39(1):195–259, 2009. doi:10.1137/ 070699652.

[18] Constantinos Daskalakis and Christos H. Papadimitriou. Continuous local search. In Proceedings of the Twenty-Second Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2011, San Francisco, California, USA, January 23-25, 2011, pages 790–804, 2011. doi:10.1137/1.9781611973082.62.

[19] Xiaotie Deng, Jack R. Edmonds, Zhe Feng, Zhengyang Liu, Qi Qi, and Zeying Xu. Understanding PPA-Completeness. In Ran Raz, editor, 31st Conference on Computational Complexity (CCC 2016), volume 50 of Leibniz International Proceedings in Informatics (LIPIcs), pages 23:1–23:25, Dagstuhl, Germany, 2016. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. doi:10.4230/LIPIcs.CCC.2016.23.

[20] Aris Filos-Ratsikas and Paul W. Goldberg. Consensus halving is ppa-complete. In Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2018, Los Angeles, CA, USA, June 25-29, 2018, pages 51–64, 2018. doi:10.1145/3188745.3188880.

[21] Aris Filos-Ratsikas and Paul W. Goldberg. The complexity of splitting necklaces and bisecting ham sandwiches. In STOC (to appear), 2019. arXiv:1805.12559.

[22] C. Goldberg and D. West. Bisection of circle colorings. SIAM Journal on Algebraic Discrete Methods, 6(1):93–106, 1985. doi:10.1137/0606010.

[23] Mika Göös, Pritish Kamath, Robert Robere, and Dmitry Sokolov. Adventures in monotone complexity and TFNP. In 10th Innovations in Theoretical Computer Science Conference, ITCS 2019, January 10-12, 2019, San Diego, California, USA, pages 38:1–38:19, 2019. doi:10.4230/LIPIcs.ITCS.2019.38.

[24] Michelangelo Grigni. A sperner lemma complete for ppa. Information Processing Letters, 77(5-6):255–259, 2001.

[25] Alexandros Hollender. The classes PPA-k: Existence from arguments modulo k. In Ioannis Caragiannis, Vahab Mirrokni, and Evdokia Nikolova, editors, Web and Internet Economics, pages 214–227, Cham, 2019. Springer International Publishing.

[26] Emil Jerábek. Integer factoring and modular square roots. J. Comput. Syst. Sci., 82(2):380–394, 2016. doi:10.1016/j.jcss.2015.08.001.

[27] Alan S. Johnson. Reductions and propositional proofs for total NP search problems. UC San Diego Electronic Theses and Dissertations, 2011. URL: https://escholarship.org/uc/ item/89r774x7.

[28] David S. Johnson, Christos H. Papadimitriou, and Mihalis Yannakakis. How easy is local search? J. Comput. Syst. Sci., 37(1):79–100, 1988. doi:10.1016/0022-0000(88)90046-3.

[29] Ilan Komargodski, Moni Naor, and Eylon Yogev. White-box vs. black-box complexity of search problems: Ramsey and graph property testing. Journal of the ACM (JACM), 66(5):34, 2019.

[30] Pravesh K Kothari and Ruta Mehta. Sum-of-squares meets nash: lower bounds for finding any equilibrium. In Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, pages 1241–1248. ACM, 2018.

[31] Donald L. Kreher and Douglas R. Stinson. Combinatorial Algorithms: Generation, Enumeration, and Search, volume 7 of Discrete Mathematics and Its Applications. CRC Press, 1998.

[32] Nimrod Megiddo and Christos H. Papadimitriou. On total functions, existence theorems and computational complexity. Theor. Comput. Sci., 81(2):317–324, 1991. doi:10.1016/ 0304-3975(91)90200-L.

[33] Christos H. Papadimitriou. On the complexity of the parity argument and other inefficient proofs of existence. J. Comput. Syst. Sci., 48(3):498–532, 1994. doi:10.1016/S0022-0000(05) 80063-7.

[34] Christian Reiher. On kemnitz'conjecture concerning lattice-points in the plane. The Ramanujan Journal, 13(1-3):333–337, 2007.

[35] Aviad Rubinstein. Settling the complexity of computing approximate two-player nash equilibria. In IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA, pages 258–265, 2016.

[36] Katerina Sotiraki, Manolis Zampetakis, and Giorgos Zirdelis. Ppp-completeness with connections to cryptography. In 59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, Paris, France, October 7-9, 2018, pages 148–158, 2018. doi: 10.1109/FOCS.2018.00023.

[37] Ewald Warning. Bemerkung zur vorstehenden arbeit von herrn chevalley. Abh. Math. Sem. Univ. Hamburg, 11:76–83, 1936.